TradeVision — Project Plan v2.0

Algorithmic Trading Simulator & Backtesting Engine

Version	Author	Date	Status
v2.0	Kavindu Jayakody	2026-10-10	In development — Phase 3 (Week 6)

> v2.0 replaces v1.0. It keeps the original vision and stack, records what has actually been built, closes the gaps found in a review of v1.0 against the codebase, and re-plans the remaining weeks. Section 2 lists every change.

---

1. Executive Summary

TradeVision is a full-stack algorithmic trading simulator and backtesting platform. Traders write strategies in JavaScript, test them against historical market data, simulate live trading with virtual funds, and get AI-written analysis of the results — all without risking real money.

The project demonstrates production-grade engineering: microservices, distributed job processing, real-time streaming, time-series data, safe execution of untrusted user code, and AI integration.

Vision: democratize algorithmic trading with a professional-grade strategy testing platform that rivals enterprise tools.

Key objectives

- Backtest years of market data in seconds.
- Stream live prices to dashboards over WebSockets.
- Run user-written strategy code safely, in an isolated sandbox.
- Provide AI-powered analysis and improvement suggestions.
- Simulate paper trading against live prices with a virtual portfolio.
- Ship a live, publicly accessible product with full documentation.

---

2. What Changed from v1.0

A review of v1.0 against the codebase found gaps in four areas. Each is now addressed in this plan.

2.1 Critical gaps (security and correctness)

Gap in v1.0	Resolution in v2.0
User strategy code runs on the server with no isolation — a remote code execution risk.	New **Strategy Runtime** (Section 5): every strategy runs in an `isolated-vm` sandbox with memory, CPU-time and wall-clock limits, and no access to network, filesystem or environment.

Gap in v1.0	Resolution in v2.0
No defined contract for strategy scripts. Templates recompute indicators over the full history on every candle ($O(n^2)$), which cannot meet the "1 year of 1-minute data in < 10 s" target.	Defined `onCandle(ctx)` contract with incrementally computed indicators (Section 5.2).
Nothing prevents a strategy from reading future candles (look-ahead bias).	The runtime exposes only candles up to the current index (Section 5.3).
Backtest results do not record which strategy version produced them.	`backtest_results.strategy_version` added (Section 8).

2.2 Work required by v1.0 requirements but never scheduled

Missing item	Now scheduled
App shell: persistent navigation between pages	Week 7
Frontend auth pages (login, register, verify email, forgot password) and route protection	Week 7
Public landing page (the demo URL currently redirects to `/market`)	Week 7
Strategy list page wired to the real API, status toggle, archive	Week 6
Notification Service (port 3006) — listed in v1.0 but never scheduled	Week 11
Profile, settings, API key management, account deletion with export (FR-AUTH-003)	Week 16
Public strategy browsing and sharing (FR-STRATEGY-002)	Week 16
Email provider for verification emails	Week 7 (decision), Week 11 (templates)

2.3 Schema and API gaps

- v1.0 defined only 4 tables. Paper trading, watchlists, alerts, API keys and strategy versions had no schema. **Section 8 now defines all tables.**
- `backtest_results` could not store what FR-BACKTEST-003 requires (Sortino, trade log, equity curve, monthly returns, benchmark) or the run configuration (slippage, commission, position sizing). **Fixed.**
- v1.0 contradicted itself: Section 3.3 put backtest results in MongoDB, Section 6.1 in PostgreSQL. **Resolved:** a summary row in PostgreSQL, the heavy per-run detail in MongoDB (Section 8.3).
- v1.0 documented ~19 endpoints against a target of 30+. **Section 9 now documents all endpoints, including market data, watchlist, alerts, portfolio, AI and profile.**
- TimescaleDB hypertables cannot be created by Prisma. **A raw SQL migration step is now specified** (Section 8.2).

2.4 Decisions that drifted from v1.0

v1.0 said	Reality / decision
Alpaca Markets for market data	**Twelve Data** is what `market-data-service` uses. Rate-limit and coverage risks updated accordingly.

v1.0 said	Reality / decision
Forex majors supported (FR-MARKET-001)	Kept only if the data provider plan covers it; otherwise removed from scope.
Unadjusted historical prices	**Split- and dividend-adjusted** prices are required for stock backtests (Section 6.2).
Tests in Week 14, CI in Week 15	**CI from Week 7**; tests written alongside features.
Production hosting for Postgres/TimescaleDB unspecified	**Open decision** (Section 13).
—	`apps/docs` is the unused Turborepo starter app and shares port 3001 with the auth service. **Remove it.**

---

3. Current Status (as of 2026-10-10)

Status is taken from the repository, not from memory.

Phase	Item	Status
1 — Foundation	Turborepo monorepo, TypeScript, ESLint, Prettier	Done
	Docker Compose infrastructure	Done
	Prisma schema and migrations	Done
	Auth service: register, login, refresh, logout, verify email, `/me`, OAuth routes	Done (backend)
	API gateway	Done
	Frontend auth pages	**Not started** — moved to Week 7
2 — Market Data	Market data service (Twelve Data)	Done
	WebSocket live prices	Done
	Candlestick chart (Lightweight Charts)	Done
	Watchlist (user-managed) and price alerts	**Partial** — watchlist is hard-coded, no alerts
3 — Strategy Builder	Indicator package (SMA, EMA, RSI, MACD, Bollinger, ATR) with tests	Done
	Strategy CRUD, clone, versions, restore (API)	Done
	Light/dark theming, design tokens	Done
	Monaco editor on the create page	Done
	Strategy list page	**Partial** — UI built on mock data
	Template gallery and clone (UI)	**Not started**
	Version history and restore (UI)	**Not started**

4. System Architecture

4.1 Services

Service	Technology	Responsibility	Port	Status
API Gateway	Node.js, Express	Routing, auth verification, rate limiting	3000	Built
Auth Service	Node.js, JWT, Passport	Registration, login, OAuth, sessions	3001	Built
Market Data Service	Node.js, WebSocket	Live prices, historical OHLCV	3002	Built
Backtest Service	Node.js, BullMQ	Strategy CRUD, backtest jobs, results	3003	Partial
Paper Trading Service	Node.js, Redis	Virtual portfolios, live simulation	3004	Planned
AI Analysis Service	Node.js, LLM API	Strategy analysis, risk scoring	3005	Planned
Notification Service	Node.js, Socket.io	Alerts, job status, in-app notifications	3006	Planned
Frontend	Next.js	Dashboard and all user-facing pages	3007	Partial

4.2 Communication

Pattern	Used for	Technology
REST	CRUD between client and services	Express, Axios
WebSocket	Live prices, live portfolio	Socket.io
Message queue	Backtest jobs	BullMQ on Redis
Pub/Sub	Cross-service events (alert fired, backtest finished)	Redis Pub/Sub
SSE	Backtest progress to the browser	Server-Sent Events

4.3 Trust boundary (new)

Services identify the user from headers set by the gateway after it verifies the JWT. That is only safe if **nothing but the gateway can reach the services**:

- Internal services bind to a private Docker network; only the gateway and frontend are exposed.
- The gateway strips any incoming identity headers from client requests before setting its own.
- In production, services are reachable only through the gateway's private network.

4.4 Streaming through the gateway (new)

- WebSocket connections require `Upgrade` / `Connection` header forwarding in both the gateway proxy and Nginx.
- SSE requires response buffering disabled (`proxy_buffering off` , `X-Accel-Buffering: no`) and long read timeouts.

5. Strategy Runtime (new)

This is the core of the product and the largest risk in v1.0.

5.1 Isolation

Strategy code is untrusted. It runs only inside an `isolated-vm` isolate, never in the host process.

Limit	Value
Memory per isolate	128 MB
CPU time per backtest	30 s
Wall-clock time per backtest	60 s
Network, filesystem, `process`, `require`	Not available

- The **entire backtest loop runs inside the isolate.** Candle data is passed in once; the trade list comes back once. Crossing the isolate boundary per candle is too slow.
- `isolated-vm` is a native module: the backtest service Docker image must include build tools.
- Fallback if `isolated-vm` is not viable: a dedicated worker process per job with no network access, killed on timeout.
- `vm2` must not be used; it was abandoned after unfixable sandbox escapes.

5.2 Script contract

A strategy defines one function that the runtime calls once per candle:

```
function onCandle(ctx) {
  const fast = ctx.sma(10);   // value at the current candle, computed incrementally
  const slow = ctx.sma(20);

  if (fast > slow && !ctx.position) ctx.buy();
  if (fast < slow && ctx.position) ctx.sell();
}
```

`ctx` member	Meaning
`ctx.candle`	Current candle `{ time, open, high, low, close, volume }`
`ctx.history(n)`	The last `n` candles, current included — never future candles
`ctx.sma(p)`, `ctx.ema(p)`, `ctx.rsi(p)`, `ctx.macd(f, s, sig)`, `ctx.bollinger(p, k)`, `ctx.atr(p)`	Indicator value at the current candle
`ctx.position`	`{ size, entryPrice }` or `null`
`ctx.buy(qty?)`, `ctx.sell(qty?)`	Place an order, filled at the next candle's open plus slippage

`ctx` member	Meaning
`ctx.log(msg)`	Write to the run log

Indicators are computed incrementally (O(1) per candle) by the runtime from `packages/indicators`, so a full run is O(n).

Week 6 impact: `STRATEGY_TYPES` in `apps/web/lib/utils.ts` (the Monaco type hints), `DEFAULT_CODE` and the four templates in `strategyTemplates.ts` must move to this contract.

5.3 Correctness rules

- No look-ahead: `ctx.history` and indicators only see candles up to the current one.
- Orders fill at the **next** candle's open, not the current close.
- Slippage and commission are applied to every fill.
- Stock data is split- and dividend-adjusted.

5.4 Reuse

Paper trading (Phase 6) runs the same runtime and the same contract against live candles, so a strategy behaves identically in backtest and simulation.

---

6. Technology Stack

6.1 Frontend

Technology	Purpose
Next.js 16 (App Router), TypeScript	Framework
Tailwind CSS 4, shadcn/ui on Radix	Styling and components
`next-themes`	Light/dark themes
Zustand	Client state
TanStack Query	Server state and caching
Lightweight Charts	Candlestick charts
Monaco Editor	Strategy editor
Recharts	Analytics charts
Socket.io client	Live data
Playwright	End-to-end tests (new)

6.2 Backend

Technology	Purpose
Node.js, Express, TypeScript	All services
BullMQ	Backtest job queue
Socket.io	WebSocket server
Twelve Data	Live and historical market data (replaces Alpaca)
`isolated-vm`	Strategy sandbox (new)
OpenAI or Anthropic API	AI analysis (decision pending)
Passport.js, JWT (RS256)	Auth
Zod	Validation
Winston	Logging
Email provider (Resend or similar)	Verification and alert emails (new, decision pending)

6.3 Data

Database	Holds
PostgreSQL (Prisma)	Users, strategies, versions, backtest summaries, portfolios, trades, watchlists, alerts
TimescaleDB	OHLCV hypertable
MongoDB	Backtest detail (trade log, equity curve), AI analysis logs
Redis	Sessions, BullMQ, Pub/Sub, price cache

6.4 DevOps

Docker Compose, GitHub Actions, Railway or Render (services), Vercel (frontend), Nginx.

---

7. Functional Requirements

v1.0 requirements FR-AUTH, FR-MARKET, FR-STRATEGY, FR-BACKTEST, FR-AI and FR-PAPER stand, with the amendments marked below.

7.1 Authentication & users

- **FR-AUTH-001–003** — unchanged.
- **FR-AUTH-004 (new): Auth UI.** Login, register, verify-email, forgot/reset-password pages; protected routes redirect to `/login`; the session refreshes transparently on 401.

7.2 Market data

- **FR-MARKET-001** — asset coverage limited to what Twelve Data supports on the chosen plan.

- **FR-MARKET-002** — historical data is split- and dividend-adjusted for stocks.
- **FR-MARKET-003** — watchlists are user-managed and persisted (currently hard-coded).

7.3 Strategy builder

- **FR-STRATEGY-001** — strategy code follows the contract in Section 5.2.
- **FR-STRATEGY-002** — list page with status filter, search, active/paused toggle, archive, clone; public browsing.
- **FR-STRATEGY-003 (new):** template gallery on the create page; selecting a template fills the form and editor.
- **FR-STRATEGY-004 (new):** version history panel on the edit page; restoring creates a new version.

7.4 Backtesting

- **FR-BACKTEST-001–003** — unchanged, plus: every result records the strategy version it ran.
- **FR-BACKTEST-004 (new):** a run that exceeds a sandbox limit fails with a clear message (e.g. "Strategy exceeded the 30 s CPU limit") instead of hanging.

7.5 AI analysis

- **FR-AI-001–003** — unchanged. Strategy code and descriptions sent to the LLM are treated as untrusted input.

7.6 Paper trading

- **FR-PAPER-001–003** — unchanged; runs on the Section 5 runtime.

7.7 Notifications (new)

- **FR-NOTIFY-001:** in-app notifications for price alerts, drawdown alerts, backtest completion and AI analysis ready.
- **FR-NOTIFY-002:** optional email delivery per notification type.

7.8 Application shell (new)

- **FR-UI-001:** persistent navigation (sidebar) linking Dashboard, Market, Strategies, Backtests, Portfolio, Settings; active route highlighted.
- **FR-UI-002:** public landing page at `/`; the app lives under authenticated routes.
- **FR-UI-003:** every data view has loading, empty and error states.
- **FR-UI-004:** layouts work from 400 px wide upward.

---

8. Database Schema

8.1 PostgreSQL tables

Existing tables are kept. New tables and columns are marked **new**.

Table	Key columns
`users`	id, email, password_hash, full_name, avatar_url, timezone, currency, is_verified
`refresh_tokens`	id, token_hash, user_id, expires_at

Table	Key columns
`auth_accounts`	id, provider, provider_id, user_id
`email_verifications`	id, token, user_id, expires_at
`strategies`	id, user_id, name, description, code, asset_symbol, timeframe, is_public, status, version
`strategy_versions`	id, strategy_id, version, code, description, created_at
`backtest_results`	id, strategy_id, **strategy_version (new)**, user_id, job_id, status, start_date, end_date, initial_capital, final_capital, total_return_pct, max_drawdown_pct, win_rate, sharpe_ratio, **sortino_ratio (new)**, **benchmark_return_pct (new)**, total_trades, **slippage_bps, commission, position_sizing, execution_mode (new)**, **detail_doc_id (new)**, **error_message (new)**
`watchlists` **new**	id, user_id, name
`watchlist_items` **new**	id, watchlist_id, symbol, position
`price_alerts` **new**	id, user_id, symbol, condition (above/below), price, is_active, triggered_at
`portfolios` **new**	id, user_id, name, starting_capital, cash, max_position_pct, daily_loss_limit_pct, drawdown_alert_pct
`portfolio_strategies` **new**	portfolio_id, strategy_id, strategy_version, is_active
`positions` **new**	id, portfolio_id, symbol, size, avg_entry_price, opened_at
`paper_trades` **new**	id, portfolio_id, strategy_id, symbol, side, qty, price, slippage, executed_at
`notifications` **new**	id, user_id, type, title, body, read_at, created_at
`api_keys` **new**	id, user_id, name, key_hash, last_used_at, created_at

8.2 TimescaleDB

`ohlcv` (time, symbol, timeframe, open, high, low, close, volume) becomes a hypertable partitioned on `time`. Prisma cannot do this, so a raw SQL migration runs after the Prisma migration:

```
CREATE EXTENSION IF NOT EXISTS timescaledb;
SELECT create_hypertable('ohlcv', 'time', if_not_exists ⇒ TRUE);
CREATE UNIQUE INDEX IF NOT EXISTS ohlcv_symbol_tf_time ON ohlcv (symbol, timeframe, time);
```

The current `Ohlcv` Prisma model uses a `cuid` primary key; the hypertable needs `time` in every unique index, so the key changes to `(symbol, timeframe, time)`.

8.3 MongoDB

Collection	Holds
`backtest_details`	trade log, equity curve, monthly returns, run log — keyed by `backtest_results.id`
`ai_analyses`	prompt, model, response, risk score, tokens used — keyed by backtest id

The PostgreSQL row stays small and listable; the large arrays live in MongoDB.

9. API Design

All routes go through the gateway under `/api`. Responses use `{ success, data?, message }`.

9.1 Auth — `/api/auth`

`POST /register` · `POST /login` · `POST /refresh` · `POST /logout` · `POST /verify-email` · `GET /me` · `GET /google` · `GET /github` · `POST /forgot-password` (new) · `POST /reset-password` (new)

9.2 Users — `/api/users` (new)

`GET /me` · `PATCH /me` · `PATCH /me/password` · `GET /me/export` · `DELETE /me` · `GET /me/api-keys` · `POST /me/api-keys` · `DELETE /me/api-keys/:id`

9.3 Market — `/api/market` (new in docs)

`GET /history?symbol&timeframe&from&to` · `GET /quote/:symbol` · `GET /symbols?search`

9.4 Watchlists & alerts (new)

`GET /api/watchlists` · `POST /api/watchlists` · `POST /api/watchlists/:id/items` · `DELETE /api/watchlists/:id/items/:symbol` · `GET /api/alerts` · `POST /api/alerts` · `DELETE /api/alerts/:id`

9.5 Strategies — `/api/strategies`

`GET /` · `POST /` · `GET /public` · `GET /:id` · `PUT /:id` · `PATCH /:id/status` (new) · `DELETE /:id` · `POST /:id/clone` · `GET /:id/versions` · `POST /:id/versions/:version/restore`

9.6 Backtests — `/api/backtest`

`POST /` · `GET /history` · `GET /:jobId/status` · `GET /:jobId/stream` (SSE) · `GET /:jobId/result` · `DELETE /:jobId`

9.7 AI — `/api/ai` (new)

`POST /analyze/:backtestId` · `GET /analysis/:backtestId` · `GET /insights/daily`

9.8 Paper trading — `/api/portfolios` (new)

`GET /` · `POST /` · `GET /:id` · `PATCH /:id` · `POST /:id/strategies` · `DELETE /:id/strategies/:strategyId` · `GET /:id/positions` · `GET /:id/trades`

9.9 Notifications — `/api/notifications` (new)

`GET /` · `PATCH /:id/read` · `PATCH /read-all`

Total: 60+ endpoints.

9.10 WebSocket events

Event	Direction	Payload
`subscribe:market`	client → server	`{ symbols }`
`price:update`	server → client	`{ symbol, price, change, volume }`
`portfolio:update`	server → client	`{ portfolioId, value, positions }`
`notification:new`	server → client	`{ id, type, title, body }`

Backtest progress uses SSE (Section 9.6), not WebSocket.

---

10. Frontend Structure

```
apps/web/app/
├── page.tsx                    # Public landing page
├── (auth)/
│   ├── login/  register/  verify-email/  forgot-password/  reset-password/
└── (app)/                      # Authenticated; shared sidebar layout
    ├── layout.tsx              # App shell: sidebar, top bar, theme toggle
    ├── dashboard/
    ├── market/
    ├── strategies/
    │   ├── page.tsx            # List
    │   ├── create/page.tsx     # Template gallery + form + editor
    │   └── [id]/page.tsx       # Edit + version history
    ├── backtests/
    │   ├── page.tsx            # History
    │   ├── new/page.tsx        # Configuration form
    │   └── [id]/page.tsx       # Progress + results
    ├── portfolio/
    ├── community/              # Public strategies
    └── settings/
```

Shared building blocks: `components/ui` (shadcn), `services/*.service.ts` (one per backend area), `lib/api.ts` (Axios with refresh-on-401), `hooks/`.

---

11. Revised Roadmap (18 weeks)

v1.0 had 16 weeks. The unscheduled work found in the review adds about two weeks.

Phase 1 — Foundation (Weeks 1–2) — Done

Monorepo, Docker, schema, auth service, gateway.

Phase 2 — Market Data (Weeks 3–4) — Mostly done

Twelve Data integration, WebSocket prices, candlestick chart. User watchlists and alerts move to Week 11.

Phase 3 — Strategy Builder (Weeks 5–6) — In progress

Week	Task	Priority
5	Indicator package, strategy CRUD API	Done
6	Monaco editor on the create page	Done
6	Define the strategy contract (Section 5.2); update editor types, default code and templates	HIGH
6	Template gallery on the create page; clone from the list	MEDIUM
6	Edit page with version history and restore	MEDIUM
6	Strategy list on the real API: search, status filter, active/paused toggle, archive	HIGH

Phase 3.5 — Frontend Foundation (Week 7) — New

Task	Priority
App shell: `(app)` route group, sidebar, active route, top bar	HIGH
Auth pages and route protection; refresh-on-401	HIGH
Public landing page	MEDIUM
Toasts, loading/empty/error states	MEDIUM
GitHub Actions CI: lint, type-check, unit tests on every PR	HIGH
Remove `apps/docs`; pick the email provider	LOW

Phase 4 — Backtesting Engine (Weeks 8–10)

Week	Task	Priority
8	BullMQ queue and worker pool; `isolated-vm` runtime with limits; tick loop with incremental indicators	HIGH
8	Sandbox escape and limit tests	HIGH
9	Metrics: return, drawdown, win rate, Sharpe, Sortino, buy-and-hold benchmark	HIGH
9	Result storage (PostgreSQL summary + MongoDB detail); SSE progress; cancel	HIGH
10	Backtest UI: configuration form, live progress, results	HIGH
10	Equity curve, trade log, monthly returns heatmap	HIGH

Phase 5 — Notifications & Alerts (Week 11) — New

Task	Priority
Notification Service: Socket.io, Redis Pub/Sub consumers	HIGH

Task	Priority
User watchlists and price alerts end to end	MEDIUM
In-app notification menu; backtest-complete notifications	MEDIUM
Email templates (verification, alerts)	LOW

Phase 6 — AI Analysis (Weeks 12–13)

Week	Task	Priority
12	AI service, prompt design, auto-trigger on backtest completion, result caching	HIGH
13	Risk score (1–10) with breakdown; AI panel on the results page	HIGH
13	Daily market insights	LOW

Phase 7 — Paper Trading (Weeks 14–15)

Week	Task	Priority
14	Portfolio service and schema; strategies run on the Section 5 runtime against live candles; slippage	HIGH
15	Portfolio dashboard: live P&L, positions, transactions, benchmark, drawdown alerts	HIGH

Phase 8 — Account & Community (Week 16) — New

Task	Priority
Profile, password change, timezone and currency	MEDIUM
API key management	LOW
Account deletion with data export	MEDIUM
Public strategy browsing and cloning	MEDIUM

Phase 9 — Hardening & Launch (Weeks 17–18)

Week	Task	Priority
17	Integration tests (Supertest) and Playwright E2E for the main flows	HIGH
17	Security review: sandbox, gateway-only network, rate limits, header stripping	HIGH
17	Load test: 1 year of 1-minute data in < 10 s; 10 parallel backtests	HIGH
18	Production deploy; observability (logs, health checks, uptime alerts)	HIGH
18	README with architecture diagram, demo video, article	HIGH

---

12. Risks & Mitigations

Risk	Probability	Impact	Mitigation
Sandbox escape lets user code reach the server	Low	Critical	`isolated-vm`, no host APIs exposed, escape tests in CI, worker containers with no secrets
Backtests too slow for the < 10 s target	Medium	High	Incremental indicators, whole loop inside the isolate, TimescaleDB range queries
Twelve Data rate limits or coverage	Medium	High	Cache all OHLCV in TimescaleDB after the first fetch; evaluate a paid tier or a second provider
Wrong results from unadjusted prices or look-ahead	Medium	High	Adjusted data, next-candle fills, unit tests with known-answer fixtures
LLM cost growth	Low	Medium	One analysis per backtest, cache results, usage alerts
Microservice complexity	High	Medium	REST first; queues only where async is needed
Services reachable without the gateway	Medium	High	Private network, header stripping, deploy check
Schedule overrun	Medium	Medium	Phases 8 and 6's daily insights are cut first

---

13. Open Decisions

Decision	Options	Needed by
Email provider	Resend, Postmark, SendGrid	Week 7
LLM provider	OpenAI, Anthropic	Week 12
Production PostgreSQL/TimescaleDB host	Timescale Cloud, self-hosted on Railway/Render	Week 17
Keep MongoDB or use PostgreSQL JSONB for backtest detail	MongoDB (as planned) or JSONB (one fewer database)	Week 9
Forex in scope	Depends on Twelve Data plan	Week 11

---

14. Success Metrics

Metric	Target
Core features implemented	Strategy builder, backtesting, AI analysis, paper trading
Services deployed and communicating	7 of 7
Documented, working endpoints	60+
Test coverage (unit + integration)	> 70%, plus E2E for main flows

Metric	Target
Frontend pages	15+
Local setup	One command (`docker compose up` + `pnpm dev`)
Backtest performance	1 year of 1-minute data in < 10 s
Live demo	Public URL on a custom domain